Live Web and LabVIEW

Experiences Building a Live Web UI with LabVIEW 2026

Inspiration

https://github.com/zeshanabdullah10/controlstack-web-bridge

```
CONTROLSTACK-WEB-BRIDGE
+---dist
|   \---assets
+---images
+---LabVIEW
+---node_modules
|   +---.bin
|   +---.tmp
|   +---.vite
|   |   \---deps
|   +---.vite-temp
|   +---@babel
|   |   +---code-frame
|   |   |   \---lib
|   |   +---compat-data
```

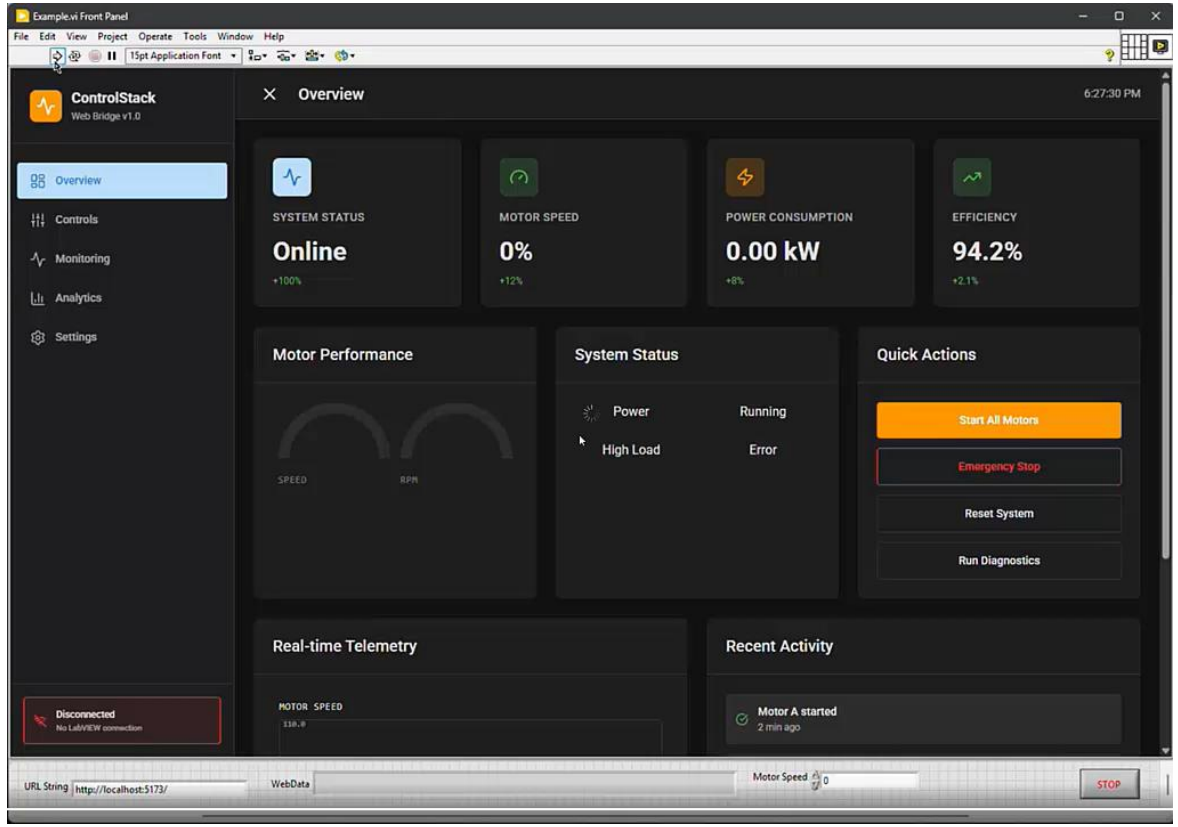
github.com/zeshanabdullah10/controlstack-web-bridge?tab=readme-ov-file

README

ControlStack Web Bridge v2.0

Modern Web-Based HMI/SCADA Interface for LabVIEW

ControlStack Web Bridge is a robust framework connecting LabVIEW applications with modern React web

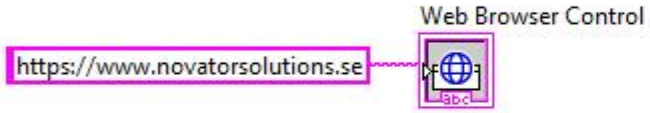


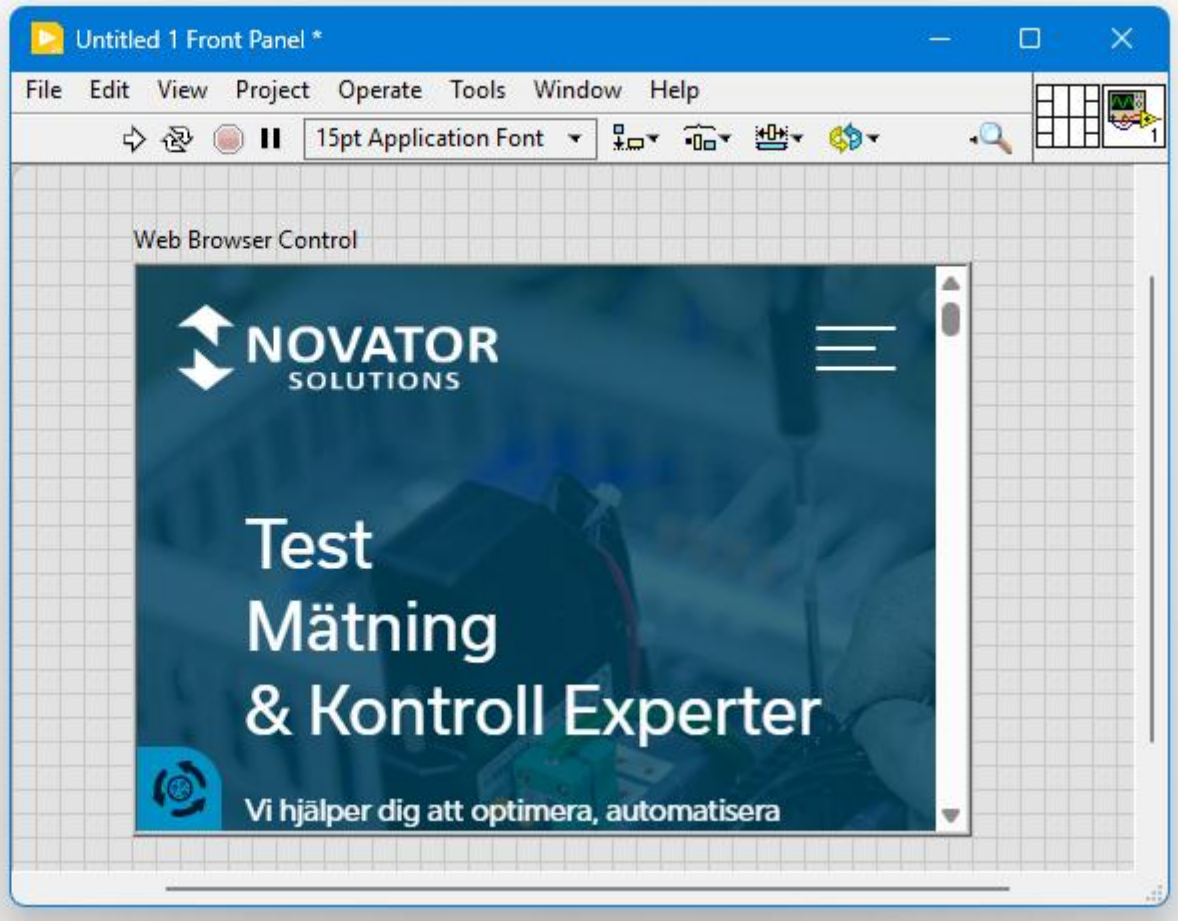
Key Features

- **Bidirectional Communication:** Seamless real-time data sync between LabVIEW and React.
- **Modern UI Components:** Includes a suite of industrial UI widgets (Gauges, Knobs, Charts, Tanks) built

Making it Work

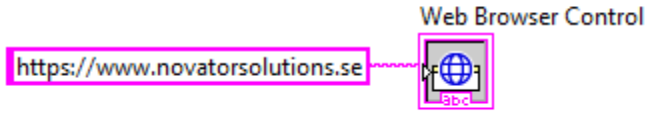
- Web Browser Front Panel Control in LabVIEW 2026



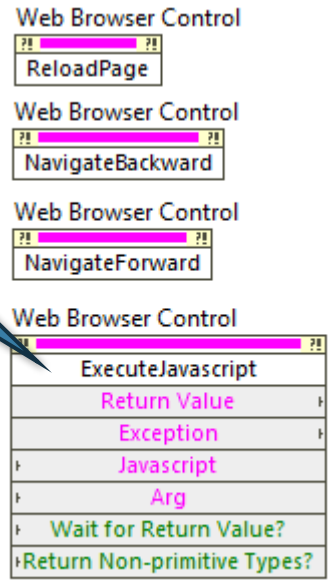


Making it Work

- Web Browser Front Panel Control in LabVIEW 2026



Invoke commands on a web page!



- Embeds Chromium Embedded Framework (CEF)
- ~~Needs Microsoft Edge WebVIEW2 (the browser engine)~~

Making it Work

- Open Web Pages
- Invoke Commands on a Web Page

- Next Step: Create your Web application that will receive the JavaScript. Options:
  - *Pure HTML + JS*
  - *Vue*
  - *Angular*
  - *Svelte*
  - *React*
  - *…*

Making it Work

With React

- Next Step: Create your Web application that will receive the JavaScript.
  - *Pure HTML + JS*
  - *Vue*
  - *Angluar*
  - *Svelte*
  - ***React (JSX)***
  - *…*

src > App.tsx > App > useEffect() callback > t > setInterval() callback

```tsx
import { useEffect, useState } from "react";
import bridge from "./lib/bridge";
import { useLabVIEW } from "./lib/useLabVIEW";

export default function App() {
  const [value, setValue] = useLabVIEW<number>("Slide", 50);
  const [connected, setConnected] = useState(false);

  useEffect(() => {
    // crude check: if refnum is set, we're "connected"
    // We'll add a getter in bridge below to make this clean.
    setConnected((bridge as any).isConnected?.() ?? false);
    const t = setInterval(() => {
      setConnected((bridge as any).isConnected?.() ?? false);
    }, 500);
    return () => clearInterval(t);
  }, []);

  return (
    <div style={{ padding: 40, fontFamily: "sans-serif" }}>
      <div>
        Status: <b>{connected ? "Connected" : "Not connected"}</b>
      </div>

      <h2>Numeric Dial</h2>

      <input
        type="range"
        min={0}
        max={100}
        value={value}
        onChange={(e) => setValue(Number(e.target.value))}
        style={{ width: 300 }}
      />

      <div style={{ marginTop: 20, fontSize: 24 }}>Value: {value}</div>
    </div>
  );
}
```

Making it Work

With React and its Infrastructure

- Open Web Pages
- Invoke Commands on a Web Page
- Create Web Application with React

→ Serve the Application in Development Environment:

- → ***Node.js**: execute JavaScript locally ← Free installer*
- → ***npm**: run tools ← From node.js installer*
- → ***Vite**: build and run server ← installed with npm*
- → ***React**: UI ← option when installing Vite*
- → ***clsx, Tailwind CSS, Framer Motion:** styling*

Making it Work

With React

- Open Web Pages
- Invoke Commands on a Web Page
- Create Web Application with React

```
npm create vite@latest my-labview-app -- --template react-ts
```

npm: app.tsx, etc. (the skeleton application)
ChatGPT: bridge.ts, useLabVIEW.ts
from GIT: Components\NumericSlider.tsx

```
npm run dev
```

```ts
export function useLabVIEW<T extends TagValue>(tag: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    if (bridge.store.has(tag)) return bridge.store.get(tag) as T;
    return initialValue;
  });
```

```ts
export default function App() {
  const [value, setValue] = useLabVIEW<number>("Slide", 50);
  const [connected, setConnected] = useState(false);
```

```ts
import { NumericSlider} from "./components/NumericSlider";
```

```ts
return (
  <div style={{ padding: 40, fontFamily: "sans-serif" }}>
    <div>
      Status: <b>{connected ? "Connected" : "Not connected"}</b>
    </div>

    <h2>Slide</h2>

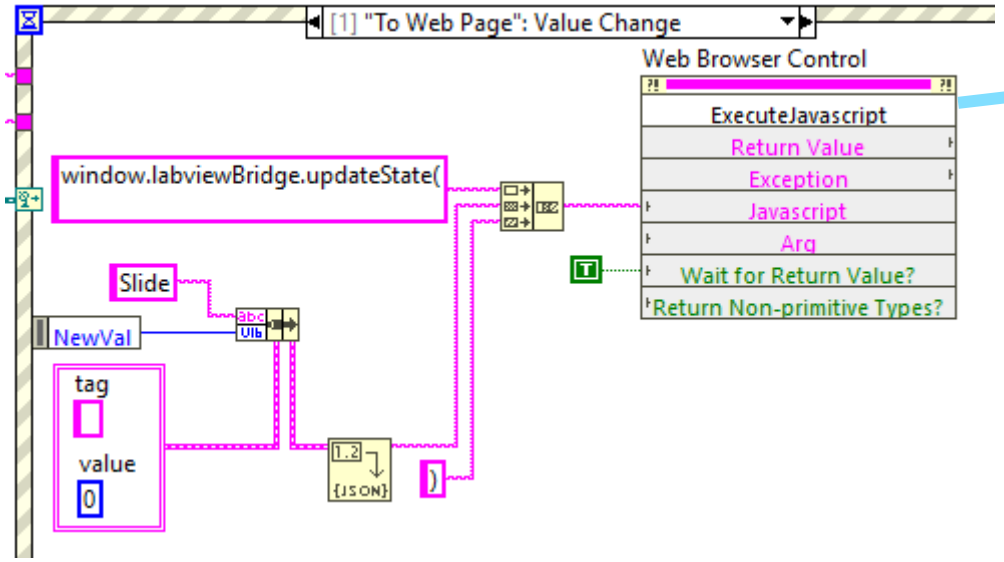
    <NumericSlider
      value={value}
      min={0}
      max={100}
      onChange={(v) => setValue(v)}
    />

    <div style={{ marginTop: 20, fontSize: 24 }}>Value: {value}</div>
  </div>
);
```

Making it Work

LabVIEW → React

- Open Web Pages
- Invoke Commands on a Web Page
- Create Web Application with React
- Serve the Application




```ts
// Method to receive updates from LabVIEW
public updateState(payload: string | LabVIEWUpdatePayload) {
  const data =
    typeof payload === "string" ? JSON.parse(payload) : payload;

  const { tag, value } = data;

  this.store.set(tag, value); // save the latest value for this tag in store

  if (this.subscribers.has(tag)) {
    this.subscribers.get(tag)!.forEach((cb) => cb(value));
  }
}
```

```ts
export function useLabVIEW<T extends TagValue>(tag: string, initialValue: T)
  const [value, setValue] = useState<T>(() => {
    if (bridge.store.has(tag)) return bridge.store.get(tag) as T;
    return initialValue;
  });
```

```ts
export default function App() {
  // subscribe to a LabVIEW tag "Slide" with an initial value of 50
  const [value, setValue] = useLabVIEW<number>("Slide", 50);
  const [connected, setConnected] = useState(false);
```

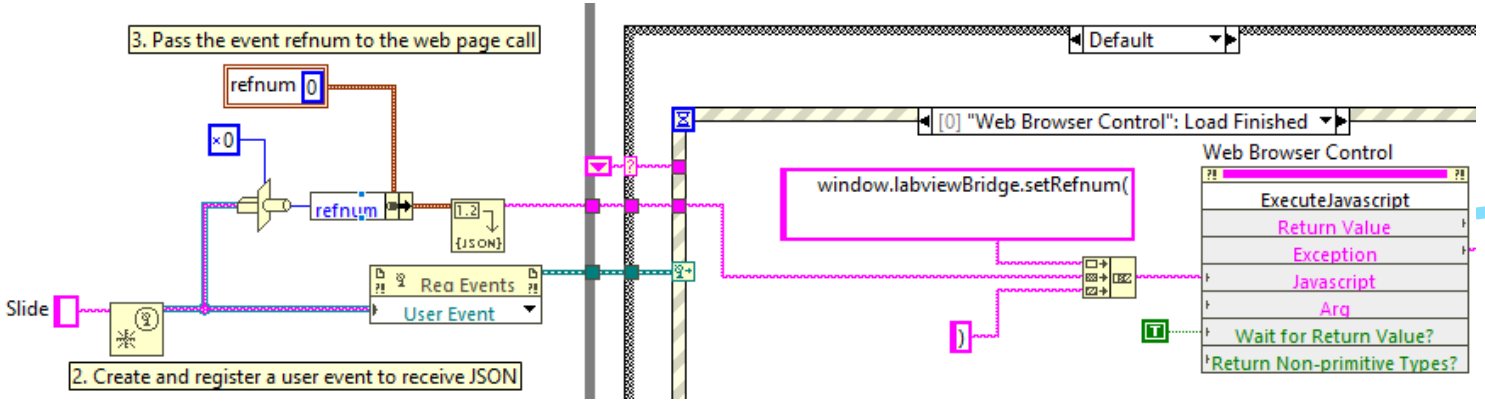
```ts
<h2>Slide</h2>

<NumericSlider
  value={value}
  min={0}
  max={100}
  onChange={(v) => setValue(v)}
/>
```

Making it Work

React → LabVIEW

- Open Web Pages
- Invoke Commands on a Web Page
- Create Web Application with React
- Serve the Application




```
public setRefnum(args: number | string | { refnum: nu
  let refnum: number | null = null;
  try {
    if (typeof args === "number") {
      refnum = args;
    } else if (typeof args === "string") {
      const trimmed = args.trim();
      if (trimmed.startsWith("{")) {
        const parsed = JSON.parse(trimmed);
        refnum = typeof parsed?.refnum === "number" ?
      } else {
        const n = Number(trimmed);
        refnum = Number.isFinite(n) ? n : null;
      }
    } else if (args && typeof args === "object") {
```

Making it Work

React → LabVIEW

- Open Web Pages
- Invoke Commands on a Web Page
- Create Web Application with React
- Serve the Application

```
<h2>Slide</h2>

<input
  type="range"
  min={0}
  max={100}
  value={value}
  onChange={(e) => setValue(Number(e.target.value))}
  style={{ width: 300 }}
/>
```

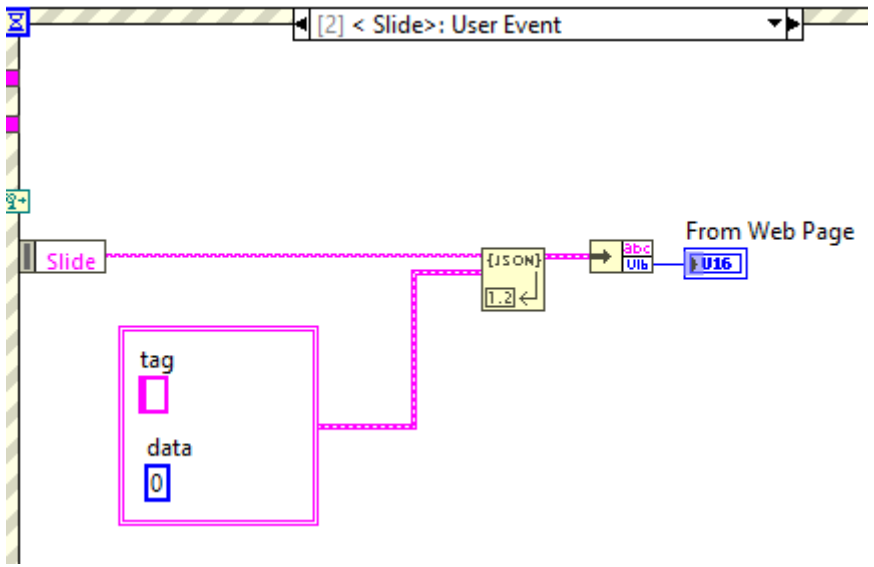
Slide

50

Value: 50

```
public sendToLabVIEW(tag: string, data: any) {
  const payload = { tag, data };
  const jsonString = JSON.stringify(payload);

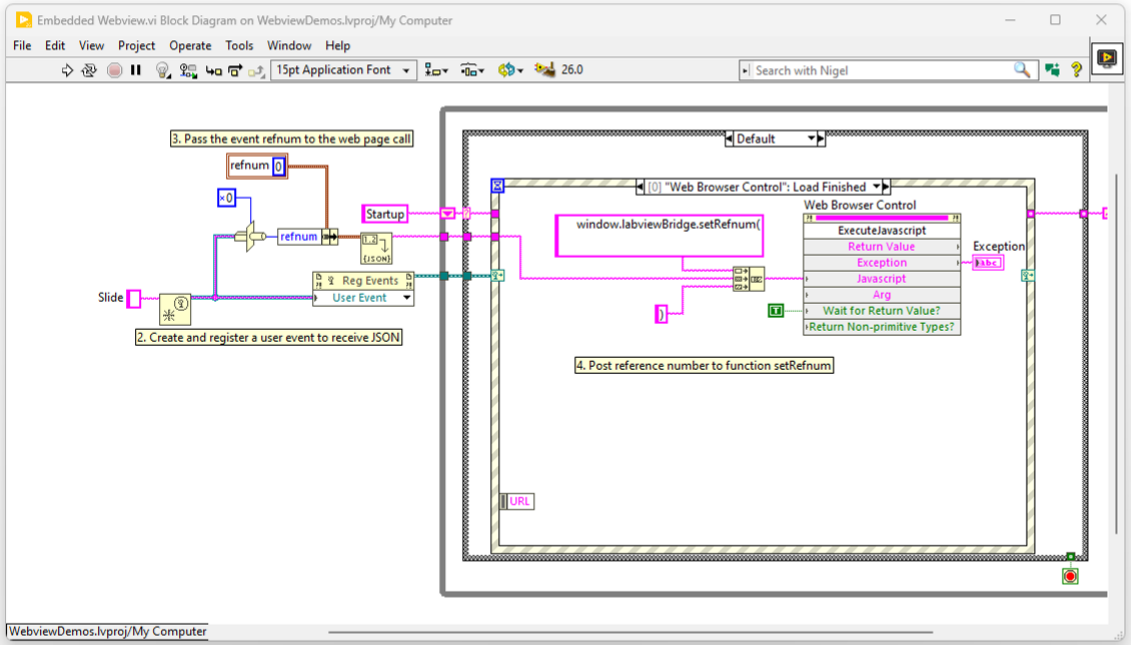
  // 1) Embedded LabVIEW WebView2 path
  if (this.refnum !== null && window.LabVIEW?.FireUserEvent) {
    window.LabVIEW.FireUserEvent(this.refnum, jsonString);
    return;
  }
```

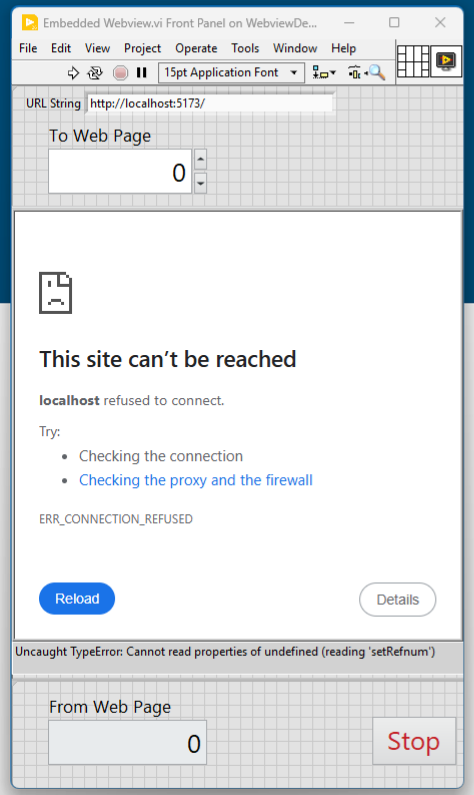




Demo 1

LabVIEW, WebVIEW2, React





Making it Work “for Real”

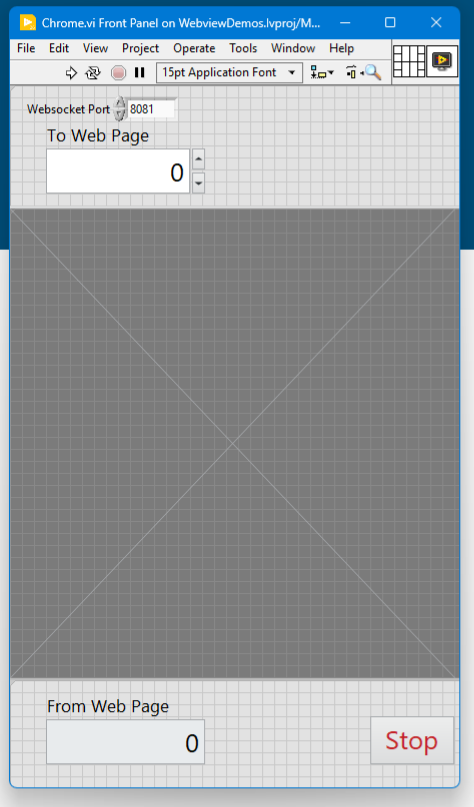
Connecting from a Web Browser

- Open Web Pages
- Invoke Commands on a Web Page
- Create Web Application with React
- Serve the Application

- Web View contains the intelligence to serve the data from LabVIEW
- A Web Page doesn’t
- Adding a Websocket server to handle the traffic from a web page
  - *React app needs to updated to support this method*

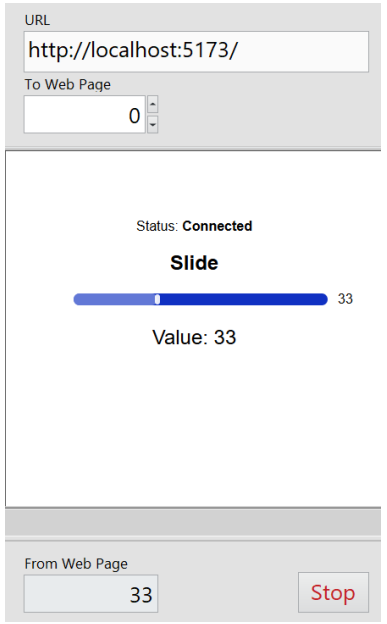
Demo 2

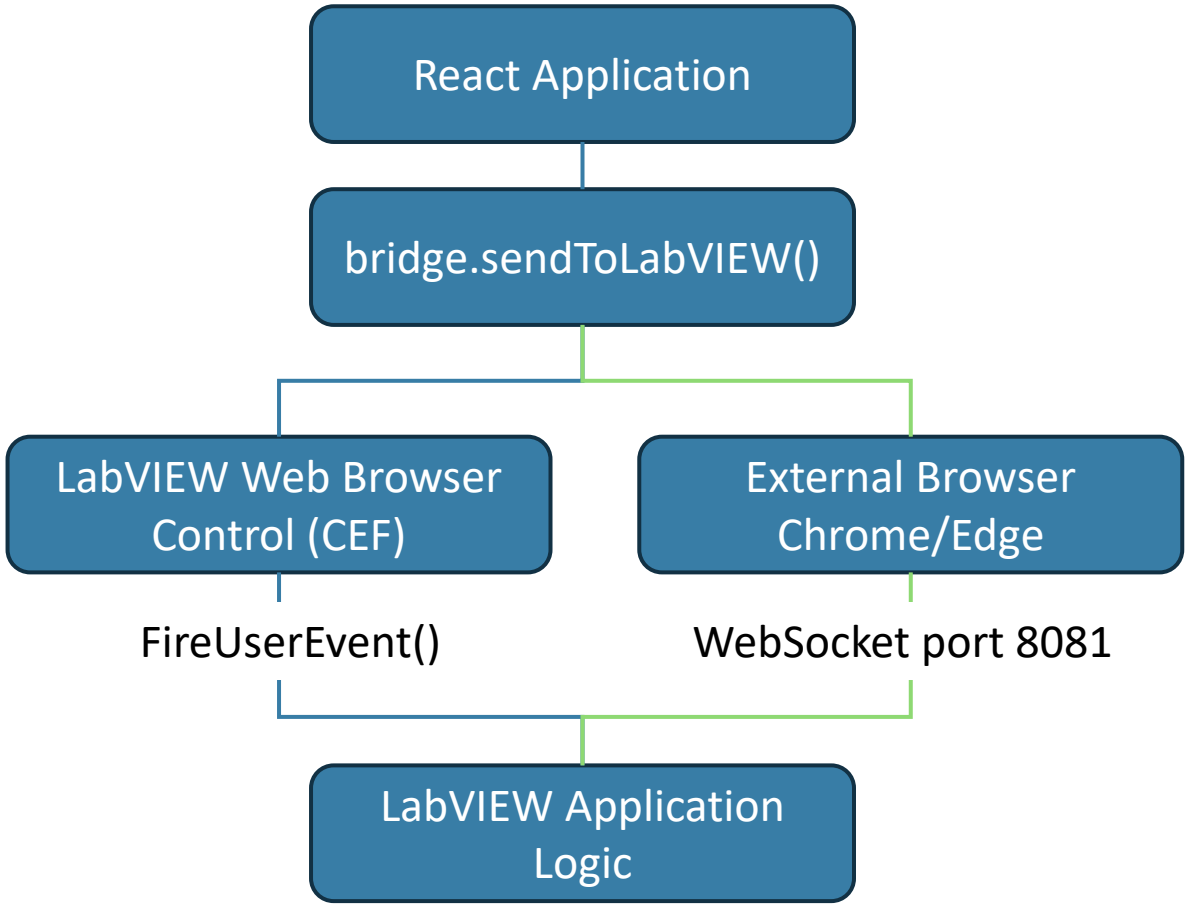
Chrome → Websocket → LabVIEW → Websocket → Chrome

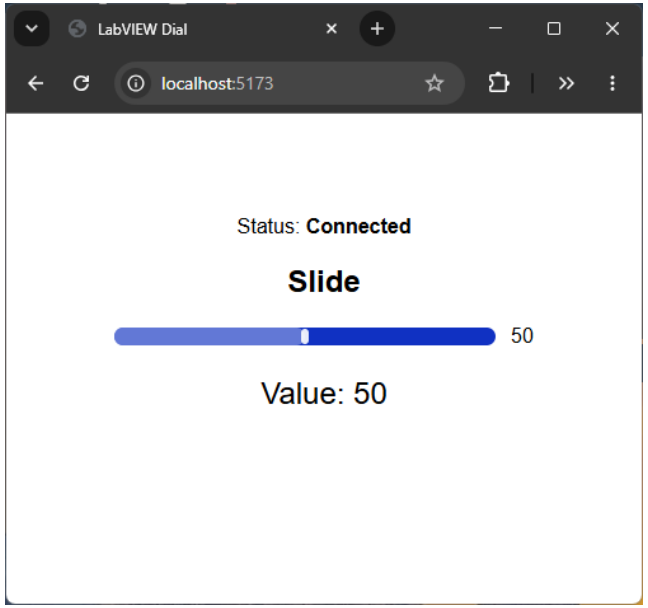


→ http://127.0.0.1:5173/

Application Overview







Questions?



Deployment

- React application (web stuff): `npm run build` generates the required components in dist/ folder
- LabVIEW Web Service (for example) serves the built application
- Build executable in LabVIEW to include LV Web Service

Deployment

- LAN Access
  - *Machine IP*
  - *Firewall*
  - *Possible authentication*
- Internet
  - *DNS, HTTPS, Certificate*
  - *Security*

Installed Software:

LabVIEW 2026

+ WebSockets 2.0.2.41 by Media Mongrels Ltd.

~~WebView2 RT https://developer.microsoft.com/en-us/microsoft-edge/webview2/?form=MA13LH#download – MS Edge browser engine~~

GIT: https://git-scm.com/install/windows (to get the example)

NODE.JS: https://nodejs.org/en/download - JavaScript runtime, includes npm package manager

VISUAL STUDIO CODE: https://code.visualstudio.com/Download# - for editing code

https://github.com/jakerol/LabVIEW-WebView-Demo

Creating a simple REACT application

```
npm create vite@latest my-labview-app -- --template react-ts

cd my-labview-app

npm install
```

Install library to use the controls:

```
npm i clsx
```